

Chapter 6: Modular Programming-Cont

Lecture 3 (live)

Mohamed M. Sabry Aly

N4-02c-92

For the shown code segment, how are parameters A and B passed?



```
MOV    R0, #0x100
LDR     R1, [R0], #4
STMFD   SP!, {R0, R1}
BL      Sub
```

0x100

A

0x104

B

A. A reference, B value

B. A value, B value

C. A reference, B reference

D. A value, B reference

Answer is D

For the shown code segment, how are parameters A and B passed?

```
MOV    R0, #0x100
LDR     R1, [R0], #4
STMFD   SP!, {R0, R1}
BL      Sub
```

0x100

A

0x104

B

The segment loads the content of memory address 0x100 to R1 → value of A

We store in the stack then the value of A and the value 0x104 (address of B)

Correct answer D: A value, B reference



Local Variables

- Subroutines often use local variables whose scope and **life span** exist only during the execution of the subroutine.
- Memory storage for these variables is **created on entry** into the subroutine and **released on exit** from subroutine.
- The **system stack** is the ideal place to create memory space for temporary variables.
- This temporary memory space is called the **stack frame**.

What is the instruction to create a stack frame for 4 local variables (32-bit each)

- A. `ADD SP, SP, #5`
- B. `SUB SP, SP, #4`
- C. `SUB SP, SP, #16`
- D. `ADD SP, SP, #16`



If we are using R11 as frame pointer, what is the instruction to store R4 to local variable c

A. STR R4, [R11,#-1]

B. STR R4, [R11,#-4]

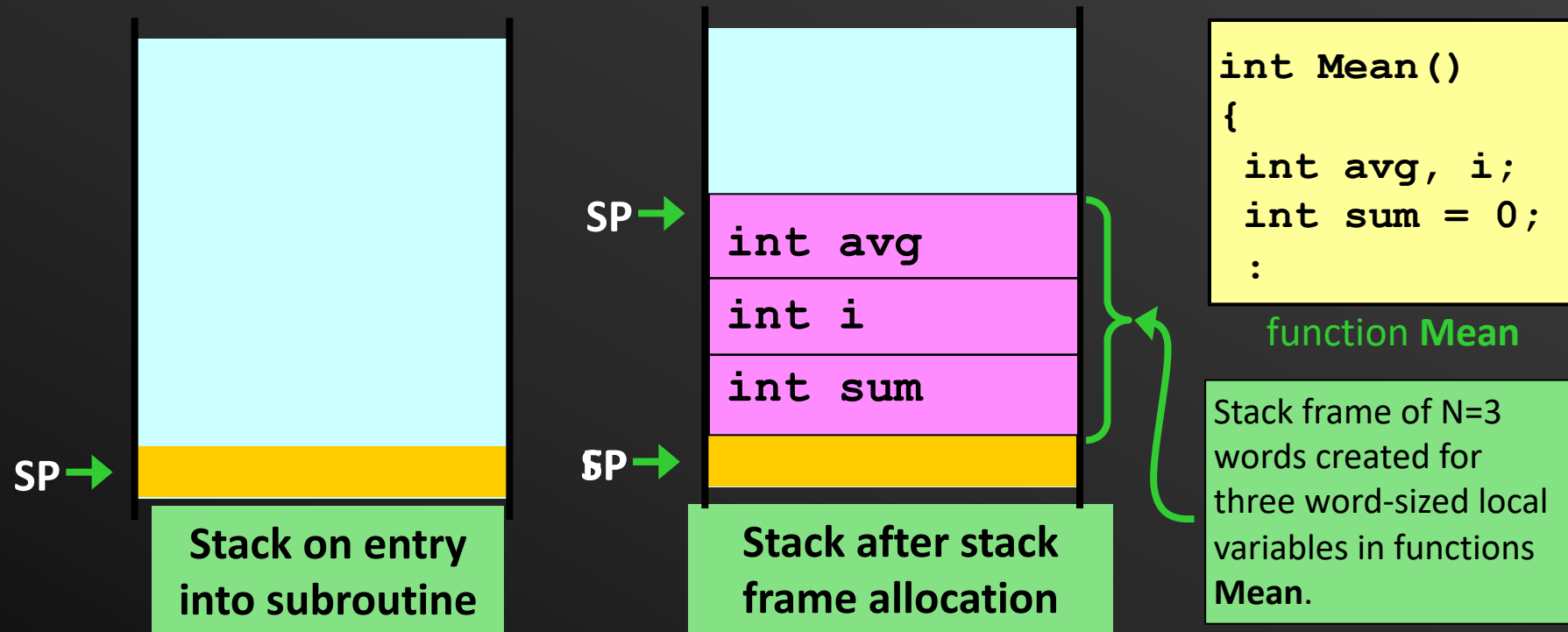
C. STR R4, [R11,#-4]!

D. STMFD R11!,{R4}

int sum
int i
int c
Saved R11

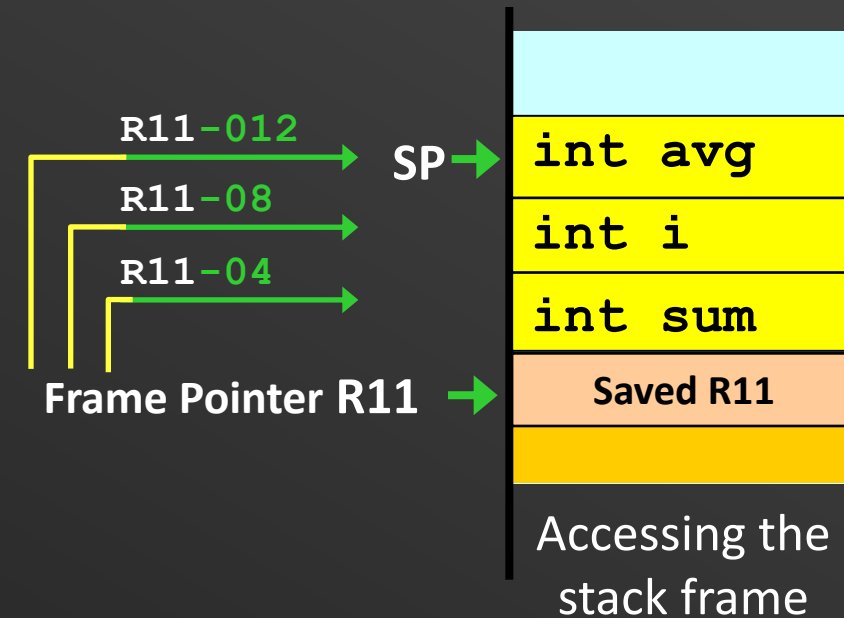
Stack Frame

- Stack frame of **N** words is created on the system stack immediately on entry into a subroutine.
- Memory space within the stack frame can be accessed using with a **frame pointer (FP)** or the stack pointer (**SP**).
- Frame pointer in ARM can be any register
 - General convention **FP** is **R11**



Accessing Stack Frame Variables Using the Frame Pointer

- Consider the use of a frame pointer register **R11**.
 - Original contents of **R11** is saved on the stack before it is used as the frame pointer.
 - Frame pointer (**R11**) now points to the saved **R11** and a stack frame is created by adding frame size **4N** to **SP**.
 - R11** is the frame reference and an appropriate negative displacement from **R11** can be used to access any stack frame variable.
 - When exiting the subroutine, the stack frame can be destroyed by adding **4N+4** to **SP** and copying the saved **R11** value on the stack back into **R11**.



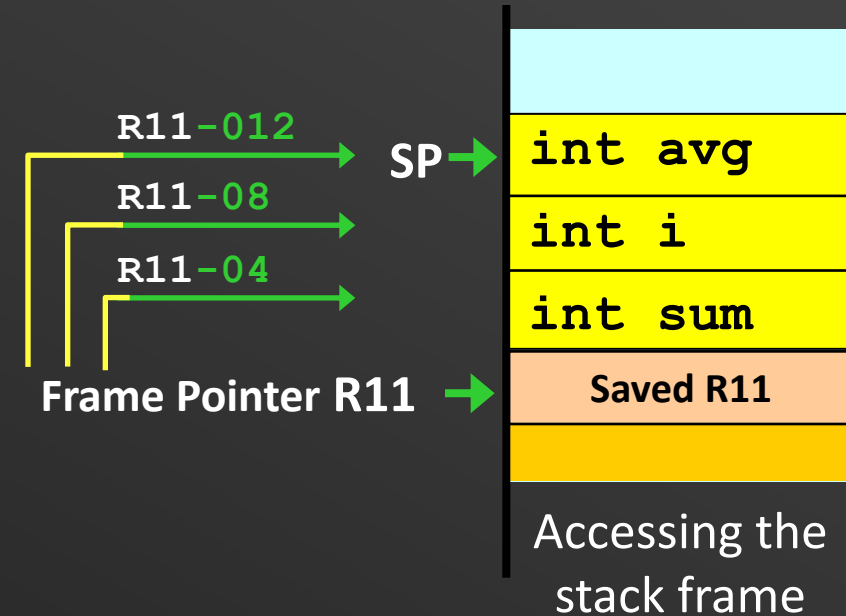
Accessing Stack Frame Variables Using the Frame Pointer

- When exiting the subroutine, the stack frame can be destroyed by adding $4N+4$ to **SP** and copying the saved **R11** value on the stack back into **R11**.

For $N=3$

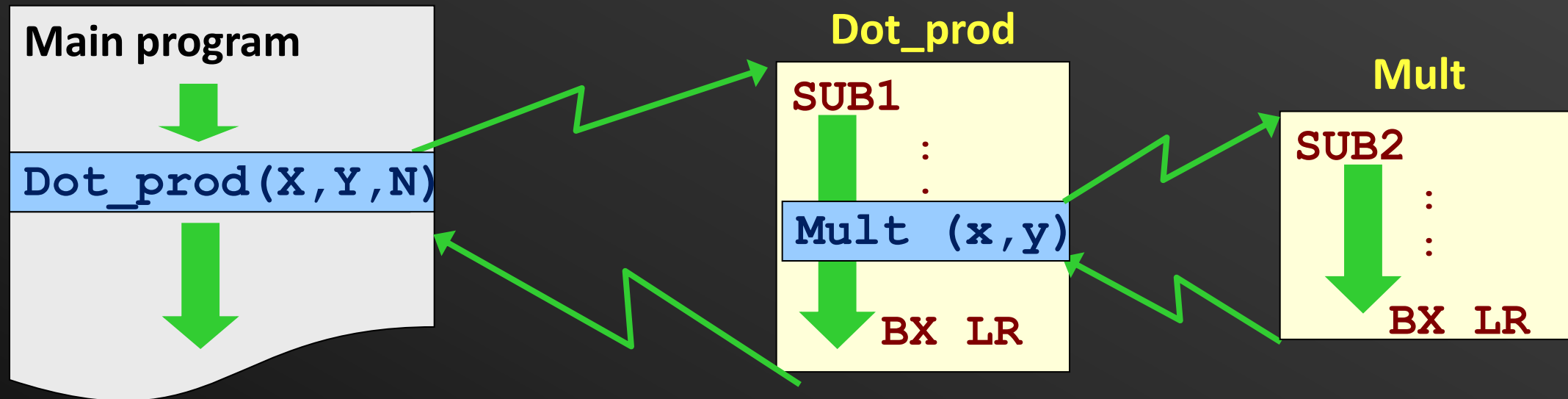
```
ADD SP, SP, #16
```

```
LDR R11, [R11]
```



Example: Dot product of two arrays

- A subroutine will call another subroutine



Example: Dot product of two arrays

- A subroutine will call another subroutine

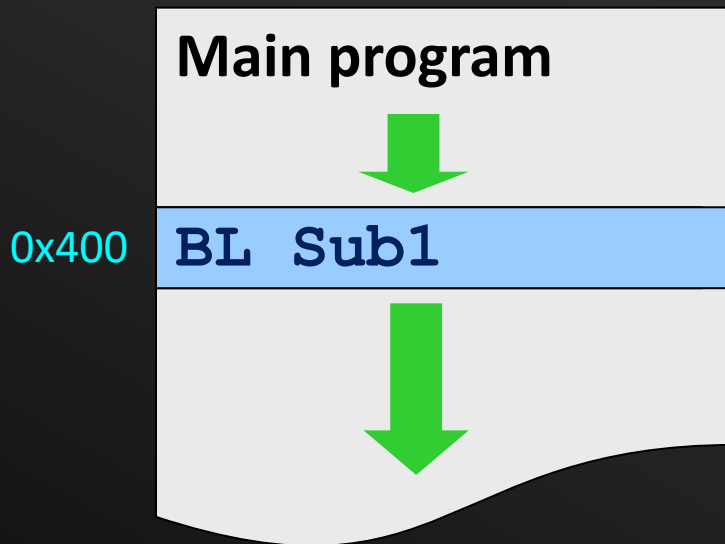
How to ensure right subroutine branch and return?

BX LR

BX LR

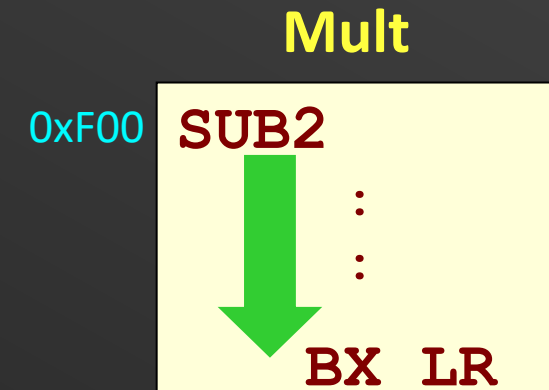
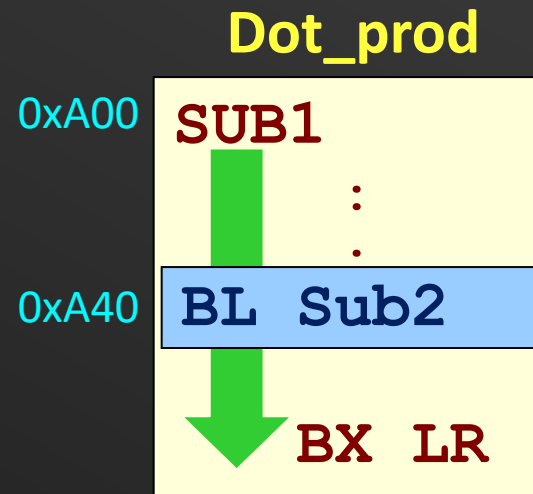
Example: Dot product of two arrays

- A subroutine will call another subroutine



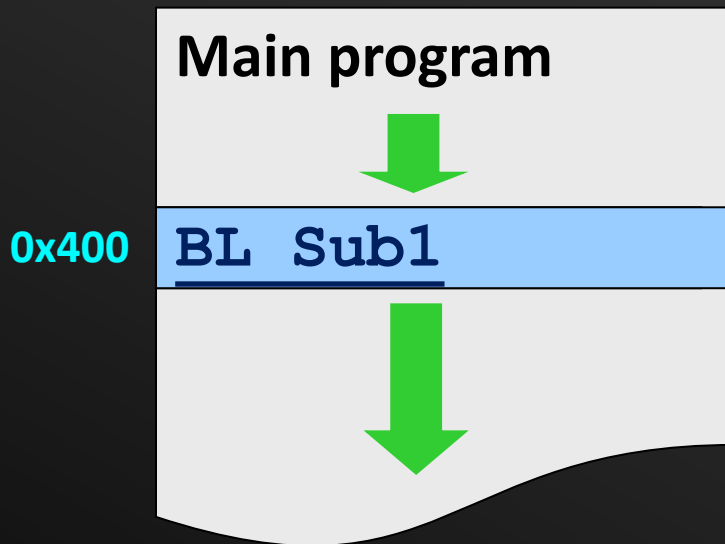
PC

LR



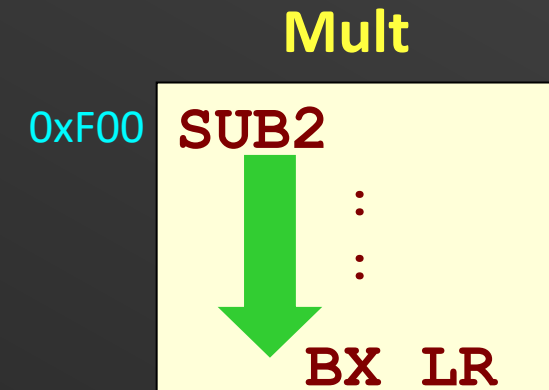
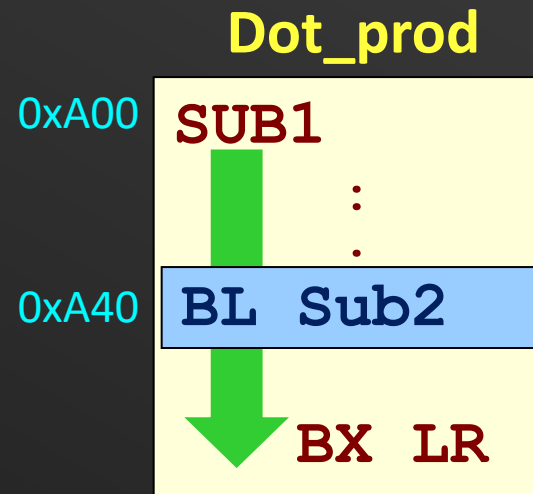
Example: Dot product of two arrays

- A subroutine will call another subroutine



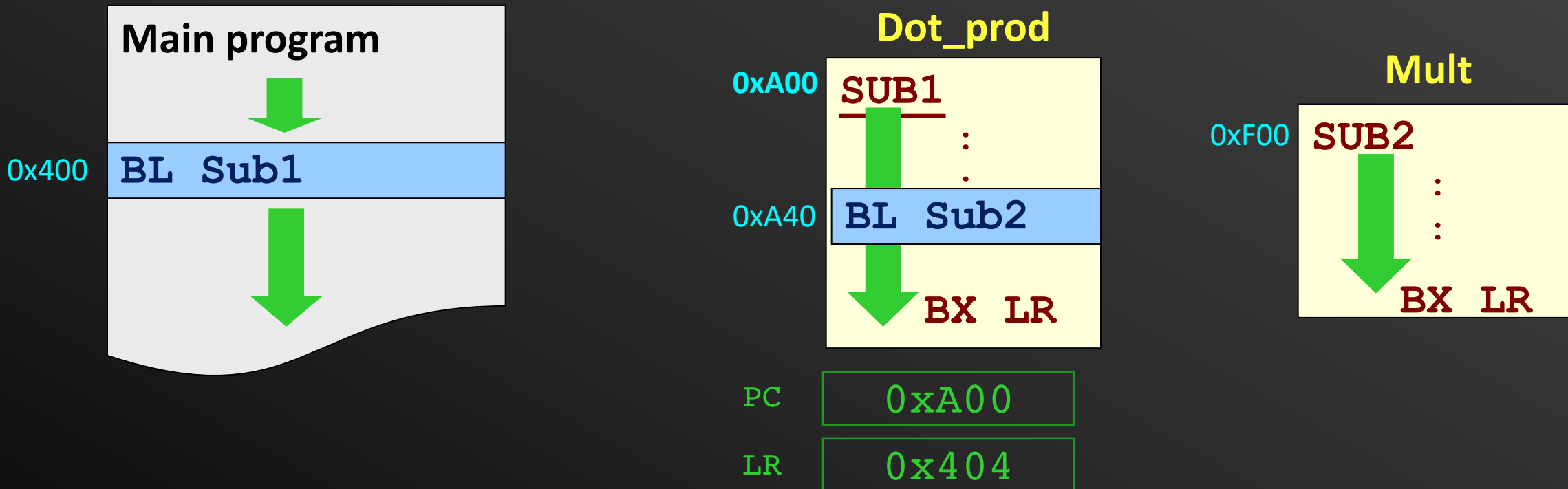
PC 0x408

LR 0x000



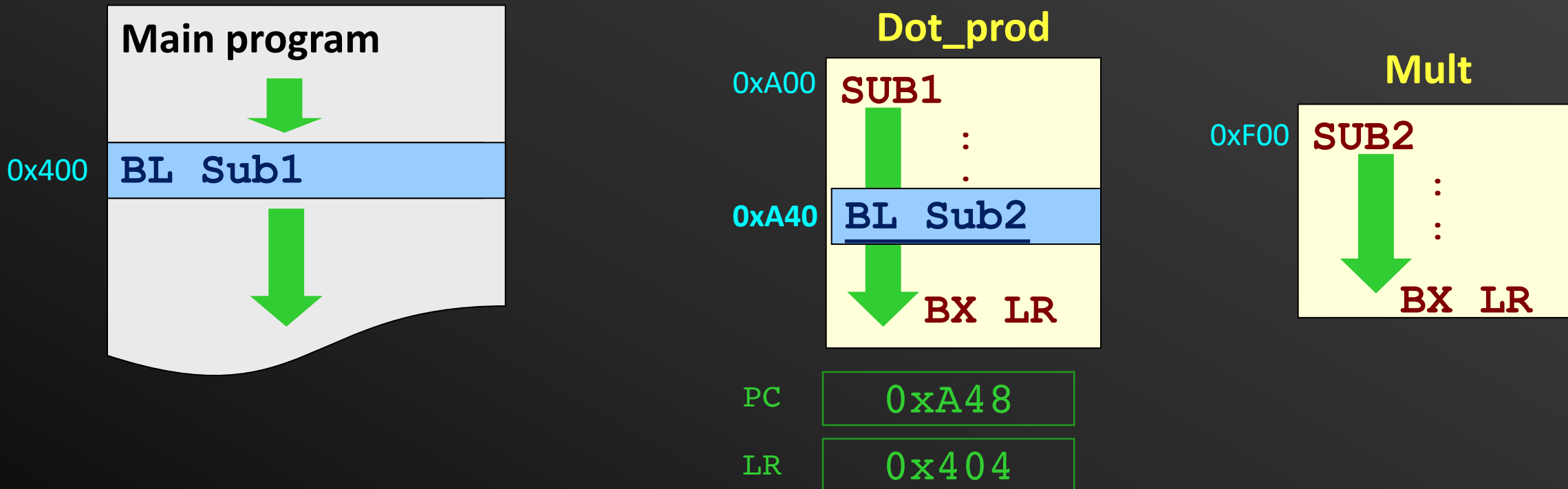
Example: Dot product of two arrays

- A subroutine will call another subroutine



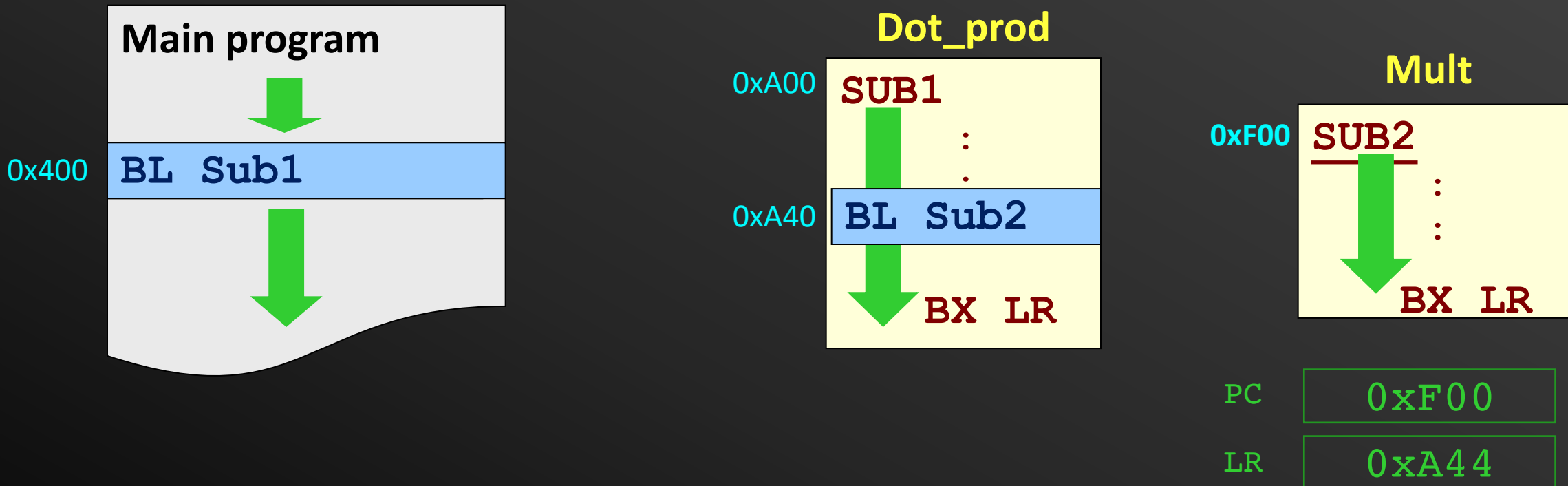
Example: Dot product of two arrays

- A subroutine will call another subroutine



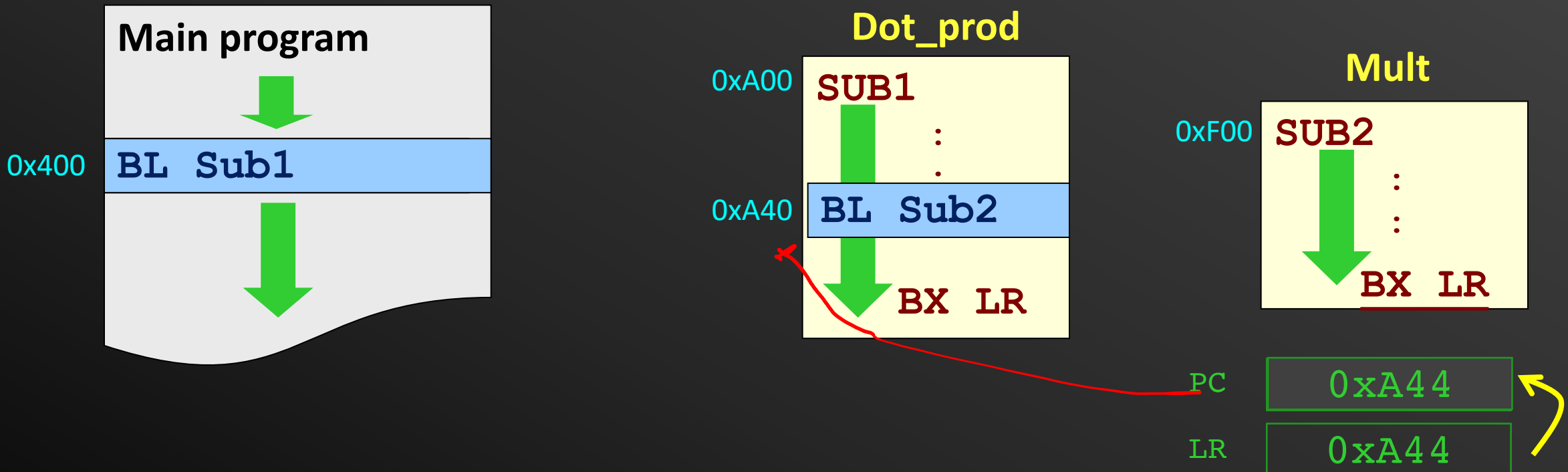
Example: Dot product of two arrays

- A subroutine will call another subroutine



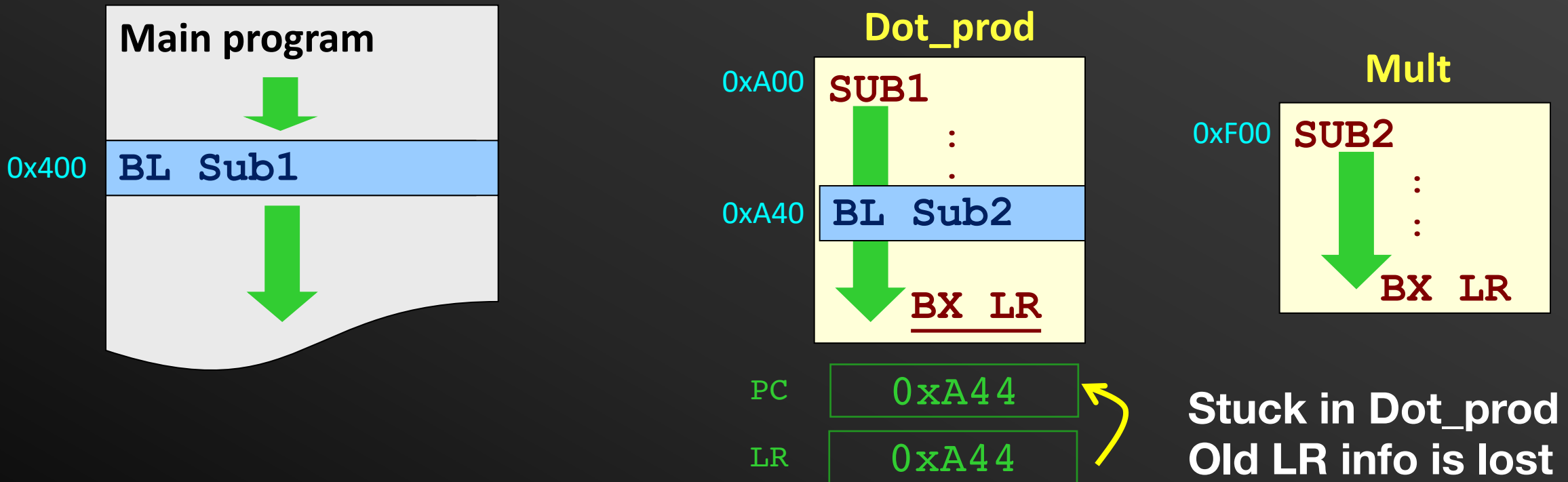
Example: Dot product of two arrays

- A subroutine will call another subroutine



Example: Dot product of two arrays

- A subroutine will call another subroutine



Support for nested Subroutine

- LR needs to be saved somewhere safe → **System stack**
- When so save it? Two options:
 - Just before any BL instruction
 - At the beginning of each subroutine

Example: Dot-product

- Write a subroutine that calculates the dot product of two vectors of **positive integers** using this equation:

$$Z = \sum_{i=0}^{N-1} X_i \times Y_i$$

- Base addresses of X (0x100), Y(0x200), the array size N(20) and Z(0x300) are passed through the stack
- The subroutine will call another subroutine to calculate the product of two numbers:
 - Numbers are passed in R0,R1, return value in R12

Multiplication subroutine

Iteratively add R0 to R12 for R1 times

Multi	MOV	R12, #0	; Clear R12
Loop	ADD	R12, R12, R0	; Add R0 to R12
	SUBS	R1, R1, #1	; Decrement R1 with 1
	BNE	Loop	
	MOV	PC, LR	; same as bx lr

SP →

0x300
20
0x200
0x100



The Dot product Subroutine

System stack when entering DotProd

```

DotProd    STMFD      SP!, {R4-R7, LR}    ; Store regs to stack
           LDR        R4, [SP, #32]       ; Read location of X
           LDR        R5, [SP, #28]       ; Read location of Y
           LDR        R6, [SP, #24]       ; Read arrays length
           MOV        R7, #0              ; Clear R7 (Sum)

Loop1      LDR        R0, [R4], #4         ; Get X[i]
           LDR        R1, [R5], #4         ; Get Y[i]

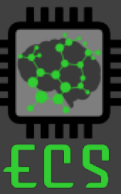
           BL         Mult                 ; Call Mult Subroutine

           ADD        R7, R7, R12          ; Add the product to R7
           SUBS       R6, R6, #1           ; Reduce the counter by 1
           BNE        Loop1               ; not 0 then repeat
           LDR        R4, [SP, #20]       ; read destination address
           STR        R7, [R4]            ; Store in destination address
           LDMFD      SP!, {R4-R7, LR}    ; Restore registers
           MOV        PC, LR              ; same as bx lr

```

The Dot product Subroutine

SP →	0x300
	20
	0x200
	0x100



System stack when entering DotProd

```
DotProd    STMFD      SP!, {R4-R7, LR}    ; Store regs to stack
           LDR        R4, [SP, #32]       ; R4
           LDR        R5, [SP, #28]       ; R5
           LDR        R6, [SP, #24]       ; R6
           MOV        R7, #0              ;
Loop1      LDR        R0, [R4], #4         ;
           LDR        R1, [R5], #4         ;
           BL         Mult                ;
           ADD        R7, R7, R12         ;
           SUBS       R6, R6, #1          ;
           BNE        Loop1              ;
           LDR        R4, [SP, #20]       ;
           STR        R7, [R4]            ;
           LDMFD      SP!, {R4-R7, LR}    ;
           MOV        PC, LR              ; same as bx lr
```

We push 5 registers to stack

SP →	R4
	R5
	R6
	R7
	LR
SP+20 →	0x300
SP+24 →	20
SP+28 →	0x200
SP+32 →	0x100

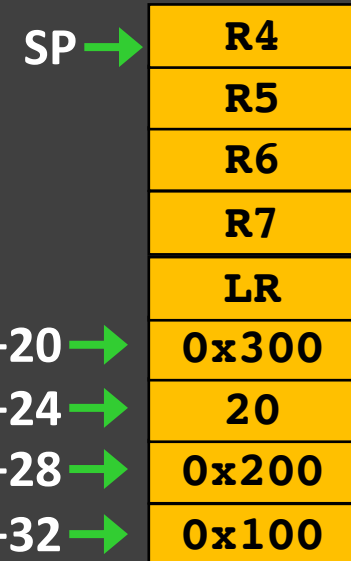
SS

The Dot product Subroutine

System stack when entering DotProd

```
DotProd    STMFD      SP!, {R4-R7, LR}    ; Store regs to stack
           LDR        R4, [SP, #32]      ; R4
           LDR        R5, [SP, #28]      ; R5
           LDR        R6, [SP, #24]      ; R6
           MOV        R7, #0             ;
Loop1      LDR        R0, [R4], #4        ;
           LDR        R1, [R5], #4        ;
           BL         Mult                ;
           ADD        R7, R7, R12         ;
           SUBS       R6, R6, #1          ;
           BNE        Loop1              ;
           LDR        R4, [SP, #20]      ;
           STR        R7, [R4]           ;
           LDMFD      SP!, {R4-R7, PC}   ;
```

We push 5 registers to stack



SS

Why is this correct?

Recursive Subroutine



- A recursive routine calls itself within its own body.
- Recursion is an elegant way to solve algorithms or mathematical expressions that have **systematic repetitions**.
(e.g. factorial $n! = n \times (n-1) \times (n-2) \dots 2 \times 1$)

```
int factorial(int n){  
    if (n<0)  
        return -1; //sanity check  
    if (n==0)  
        return 1;  
    else  
        return n * factorial(n-1);  
}
```

Recursive Subroutine

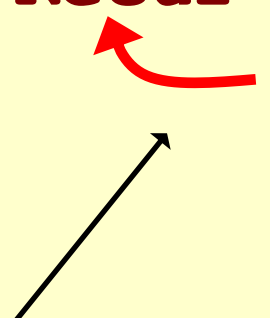
- A recursive routine calls itself within its own body.
- Recursion is an elegant way to solve algorithms or mathematical expressions that have **systematic repetitions**.
(e.g. factorial $n! = n \times (n-1) \times (n-2) \dots 2 \times 1$)

Main problem: Without saving LR, execution will be stuck

Recursive Code Example

```

Recur      :           ; Subroutine Recur
            BL Recur    ; call Recur with Recur routine
            :
            BX LR       ; return to calling program
  
```



Another problem: Some condition must be reached that allows **BL Recur** to be skipped in order to avoid infinite recursion.

Recursive Subroutine

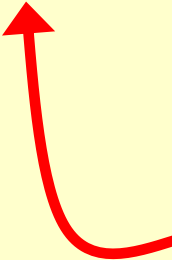
- A recursive routine calls itself within its own body.
- Recursion is an elegant way to solve algorithms or mathematical expressions that have **systematic repetitions**.
(e.g. factorial $n! = n \times (n-1) \times (n-2) \dots 2 \times 1$)

Recursive Code Example

```

Recur      STMFD SP! , { ..., LR}           ; Subroutine Recur
           :
           Condition to terminate the subroutine (branch to Done)
           :
           BL Recur                         ; call Recur with Recur routine
           :
Done        LDMFD SP! , { ..., LR}
           BX LR                           ; return to calling program

```



Length of Linked List (Pointer Chasing)

- Given this data structure, determine the length

```
int length (DS* X)
{
    int l=1;
    if (X->nxt_address==0)
        return l;
    l = 1+ length(X->nxt_address)
    return l;
}
```

Length(0x100)

X

Value (32-bit)

Nxt_address

0x100

50

0x104

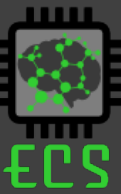
0x124

0x124

54

0x000

Example Solution



Length	STMFD	SP!, {R4-R5, LR}	; Store regs to stack	SP	R4
	LDR	R4, [SP, #12]	; read address of X		R5
	LDR	R5, [R4, #4]	; get x->nxt_address		LR
	CMP	R5, #0	; Comparing with 0		0x100
	MOVEQ	R12, #1	; Initialize l		
	BEQ	Done			
	STMFD	SP!, {R5}	; passing x->nxt_address to stack		
	BL	Length			
	LDMFD	SP!, {R5}			
	ADD	R12, R12, #1	; return 1 + length (nxt_Address)		
Done	LDMFD	SP!, {R4-R5, LR}	; Restore registers		
	MOV	PC, LR	; same as bx lr		



Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize 1
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return 1 + length (nxt_Address)
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108

SP

R4
R5
LR
0x100

R4

R5

R12

--	--	--



Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize l
Done	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return l + length (nxt_Address)
	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108

SP

R4
R5
LR
0x100

R4

0X100

R5

0X124

R12



Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize l
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return l + length (nxt_Address)
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108

SP

0x124

R4

R5

LR

0x100

R4

0X100

R5

0X124

R12



Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize 1
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	X	LDMFD	SP!,{R5}
		ADD	R12,R12,#1 ; return 1 + length (nxt_Address)
Done		LDMFD	SP!,{R4-R5,LR} ; Restore registers
		MOV	PC, LR ; same as bx lr

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108

SP

0x124

R4

R5

LR

0x100

R4

0X100

R5

0X124

R12



Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize l
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
X	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return l + length (nxt_Address)
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

SP

R4
R5
LR X
0x124
R4
R5
LR
0x100

R4	R5	R12
0X100	0X124	

0x100

0x104

0x108

0x10C

0x124

0x140

50
0x124
10
0x000
54
0x140
-1
0x108



Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize l
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	X LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return l + length (nxt_Address)
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

SP

R4
R5
LR
0x124
R4
R5
LR
0x100

R4	R5	R12
0X124	0X140	

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108



SP

0x140
R4
R5
LR
0x124
R4
R5
LR
0x100

Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize l
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
X	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return l + length (nxt_Address)
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

R4	R5	R12
0X124	0X140	

0x100

0x104

0x108

0x10C

0x124

0x140

50
0x124
10
0x000
54
0x140
-1
0x108



SP

0x140
R4
R5
LR
0x124
R4
R5
LR
0x100

```
Length      STMFD      SP!,{R4-R5,LR}      ;Store regs to stack
            LDR        R4,[SP,#12]        ; read address of X
            LDR        R5,[R4,#4]         ; get x->nxt_address
            CMP        R5, #0             ; Comparing with 0
            MOVEQ      R12,#1             ; Initialize l
            BEQ        Done
            STMFD      SP!,{R5}           ; passing x->nxt_address to stack
            BL         Length
X X LDMFD      SP!,{R5}
            ADD        R12,R12,#1         ; return l + length (nxt_Address)
Done LDMFD      SP!,{R4-R5,LR}            ; Restore registers
            MOV        PC, LR             ; same as bx lr
```

R4	R5	R12
0X124	0X140	

0x100

0x104

0x108

0x10C

0x124

0x140

50
0x124
10
0x000
54
0x140
-1
0x108



SP	R4
	R5
	LR X
	0x140
	R4
	R5
	LR X
	0x124
	R4
	R5

LR	
0x100	

Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize 1
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return 1 + length (nxt_Address)

Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

0x100	50
0x104	0x124
0x108	10
0x10C	0x000
0x124	54
	0x140
0x140	-1
	0x108

R4	R5	R12
0X124	0X140	



SP	R4
	R5
	LR
	0x140
	R4
	R5
	LR
	0x124
	R4
	R5
	LR
	0x100

Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize 1
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return 1 + length (nxt_Address)
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

R4	R5	R12
0X140	0X108	

0x100	50
0x104	0x124
0x108	10
0x10C	0x000
0x124	54
	0x140
0x140	-1
	0x108



SP

0x108
R4
R5
LR
0x140
R4
R5
LR
0x124
R4
R5
LR
0x100

```
Length      STMFD      SP!,{R4-R5,LR}      ;Store regs to stack
            LDR        R4,[SP,#12]         ; read address of X
            LDR        R5,[R4,#4]          ; get x->nxt address
            CMP        R5, #0              ; Comparing with 0
            MOVEQ      R12,#1              ; Initialize l
            BEQ        Done
            STMFD      SP!,{R5}            ; passing x->nxt_address to stack
            BL        Length
X X          LDMFD      SP!,{R5}
            ADD        R12,R12,#1          ; return 1 + length (nxt_Address)
Done         LDMFD      SP!,{R4-R5,LR}     ; Restore registers
            MOV        PC, LR              ; same as bx lr
```

R4	R5	R12
0X140	0X108	

0x100

0x104

0x108

0x10C

0x124

0x140

50
0x124
10
0x000
54
0x140
-1
0x108



SP

0x108
R4
R5
LR
0x140
R4
R5
LR
0x124
R4
R5
LR
0x100

```
Length      STMFD      SP!,{R4-R5,LR}      ;Store regs to stack
            LDR        R4,[SP,#12]        ; read address of X
            LDR        R5,[R4,#4]         ; get x->nxt_address
            CMP        R5, #0              ; Comparing with 0
            MOVEQ      R12,#1              ; Initialize l
            BEQ        Done
            STMFD      SP!,{R5}           ; passing x->nxt_address to stack
            BL         Length
X X X      LDMFD      SP!,{R5}
            ADD        R12,R12,#1         ; return l + length (nxt_Address)
Done       LDMFD      SP!,{R4-R5,LR}      ; Restore registers
            MOV        PC, LR             ; same as bx lr
```

R4	R5	R12
0X140	0X108	

0x100

0x104

0x108

0x10C

0x124

0x140

50
0x124
10
0x000
54
0x140
-1
0x108



SP	R4
	R5
	LR Pink x
	0x108
	R4
	R5
	LR X
	0x140
	R4
	R5
	LR X
	0x124
	R4
	R5
	LR
	0x100

Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize 1
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return 1 + length (nxt_Address)
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

X X X

R4	R5	R12
0X140	0X108	

0x100

0x104

0x108

0x10C

0x124

0x140

50
0x124
10
0x000
54
0x140
-1
0x108



SP	R4
	R5
	LR
	0x108
	R4
	R5
	LR
	0x140
	R4
	R5
	LR
	0x124
	R4
	R5
	LR
	0x100

Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize 1
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return 1 + length (nxt_Address)
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

X X X

0x100

0x104

0x108

0x10C

0x124

0x140

R4	R5	R12
0X108	0X000	

50
0x124
10
0x000
54
0x140
-1
0x108



SP	R4
	R5
	LR
	0x108
	R4
	R5
	LR
	0x140
	R4
	R5
	LR
	0x100

Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize 1
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return 1 + length (nxt_Address)
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

X X X

R4	R5	R12
0X108	0X000	1

0x100	50
0x104	0x124
0x108	10
0x10C	0x000
0x124	54
	0x140
0x140	-1
	0x108



SP	R4
	R5
	LR
	0x108
	R4
	R5
	LR
	0x140
	R4
	R5
	LR
	0x124
	R4
	R5
	LR
	0x100

Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize 1
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
X X X	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return 1 + length (nxt_Address)
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

0x100	50
0x104	0x124
0x108	10
0x10C	0x000
0x124	54
	0x140
0x140	-1
	0x108

R4	R5	R12
0X108	0X000	1



SP

0x108
R4
R5
LR
0x140
R4
R5
LR
0x124
R4
R5
LR
0x100

```
Length      STMFD      SP!,{R4-R5,LR}      ;Store regs to stack
            LDR        R4,[SP,#12]         ; read address of X
            LDR        R5,[R4,#4]          ; get x->nxt_address
            CMP        R5, #0              ; Comparing with 0
            MOVEQ      R12,#1              ; Initialize l
            BEQ        Done
            STMFD      SP!,{R5}            ; passing x->nxt_address to stack
            BL         Length
            LDMFD      SP!,{R5}
            ADD        R12,R12,#1          ; return l + length (nxt Address)
Done        LDMFD      SP!,{R4-R5,LR}      ; Restore registers
            MOV        PC, LR              ; same as bx lr
```

X X X

R4	R5	R12
0X140	0X108	1

0x100

0x104

0x108

0x10C

0x124

0x140

50
0x124
10
0x000
54
0x140
-1
0x108



SP

0x108
R4
R5
LR
0x140
R4
R5
LR
0x124
R4
R5
LR
0x100

```
Length      STMFD      SP!,{R4-R5,LR}      ;Store regs to stack
            LDR        R4,[SP,#12]         ; read address of X
            LDR        R5,[R4,#4]         ; get x->nxt_address
            CMP        R5, #0              ; Comparing with 0
            MOVEQ      R12,#1              ; Initialize l
            BEQ        Done
            STMFD      SP!,{R5}            ; passing x->nxt_address to stack
            BL         Length
            LDMFD      SP!,{R5}
            ADD        R12,R12,#1          ; return 1 + length (nxt_Address)
            LDMFD      SP!,{R4-R5,LR}      ; Restore registers
Done         MOV        PC, LR              ; same as bx lr
```

X X X



R4	R5	R12
0X140	0X108	1

0x100

0x104

0x108

0x10C

0x124

0x140

50
0x124
10
0x000
54
0x140
-1
0x108



SP

R4
R5
LR
0x140
R4
R5
LR
0x124
R4
R5
LR
0x100

```
Length      STMFD      SP!,{R4-R5,LR}      ;Store regs to stack
            LDR        R4,[SP,#12]        ; read address of X
            LDR        R5,[R4,#4]         ; get x->nxt_address
            CMP        R5, #0             ; Comparing with 0
            MOVEQ      R12,#1             ; Initialize l
            BEQ        Done
            STMFD      SP!,{R5}           ; passing x->nxt_address to stack
            BL         Length
X X LDMFD    SP!,{R5}
            ADD        R12,R12,#1         ; return l + length (nxt_Address)
Done LDMFD    SP!,{R4-R5,LR}              ; Restore registers
            MOV        PC, LR             ; same as bx lr
```

R4	R5	R12
0X140	0X108	1

0x100

0x104

0x108

0x10C

0x124

0x140

50
0x124
10
0x000
54
0x140
-1
0x108



SP

R4
R5
LR
0x140
R4
R5
LR
0x124
R4
R5
LR
0x100

```
Length      STMFD      SP!,{R4-R5,LR}      ;Store regs to stack
            LDR        R4,[SP,#12]        ; read address of X
            LDR        R5,[R4,#4]         ; get x->nxt_address
            CMP        R5, #0              ; Comparing with 0
            MOVEQ      R12,#1              ; Initialize 1
            BEQ        Done
            STMFD      SP!,{R5}            ; passing x->nxt_address to stack
            BL         Length
X X LDMFD      SP!,{R5}
Done ADD        R12,R12,#1                  ; return 1 + length (nxt_Address)
            LDMFD      SP!,{R4-R5,LR}      ; Restore registers
            MOV        PC, LR              ; same as bx lr
```

R4	R5	R12
0X140	0X108	2

0x100

0x104

0x108

0x10C

0x124

0x140

50
0x124
10
0x000
54
0x140
-1
0x108



SP

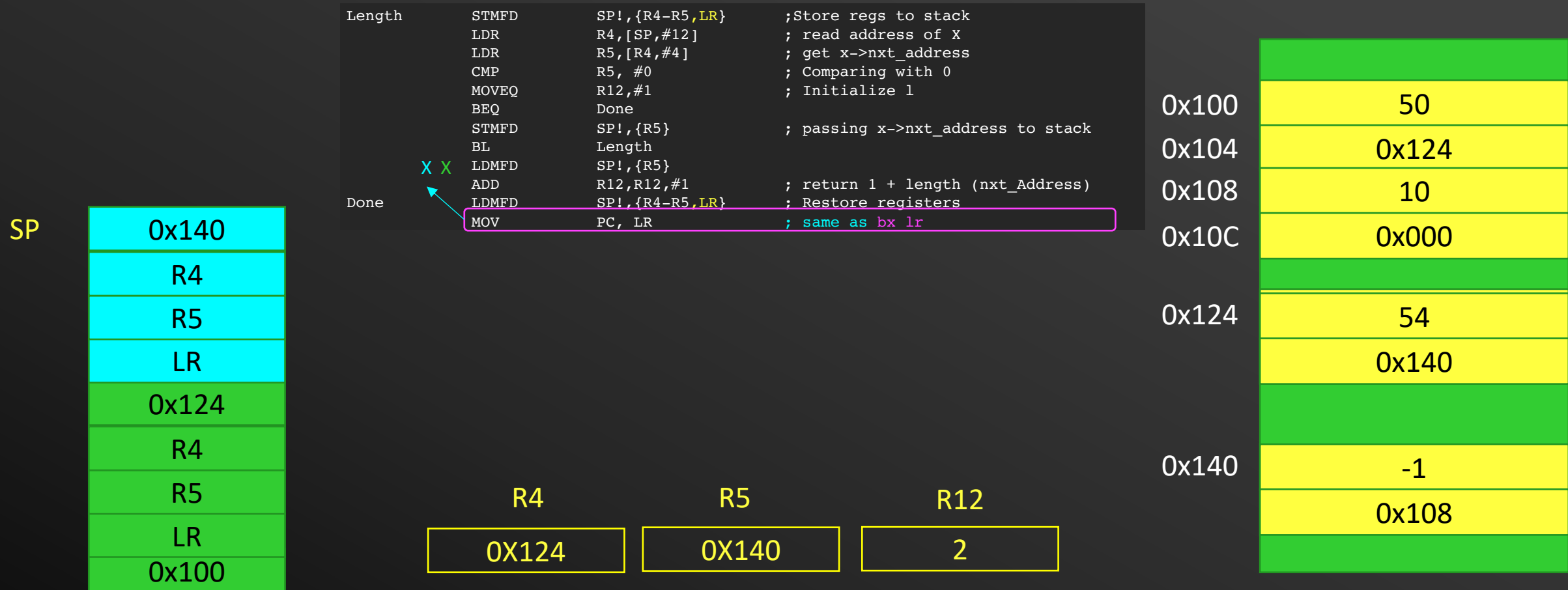
0x140
R4
R5
LR
0x124
R4
R5
LR
0x100

```
Length      STMFD      SP!,{R4-R5,LR}      ;Store regs to stack
            LDR        R4,[SP,#12]        ; read address of X
            LDR        R5,[R4,#4]         ; get x->nxt_address
            CMP        R5, #0              ; Comparing with 0
            MOVEQ      R12,#1              ; Initialize l
            BEQ        Done
            STMFD      SP!,{R5}            ; passing x->nxt_address to stack
            BL         Length
X X LDMFD      SP!,{R5}
            ADD        R12,R12,#1          ; return l + length (nxt_Address)
Done LDMFD      SP!,{R4-R5,LR}              ; Restore registers
            MOV        PC, LR              ; same as bx lr
```

R4	R5	R12
0X124	0X140	2

0x100

50
0x124
0x124
10
0x10C
0x000
0x124
54
0x140
0x140
-1
0x108





```
Length      STMFD      SP!,{R4-R5,LR}      ;Store regs to stack
            LDR        R4,[SP,#12]         ; read address of X
            LDR        R5,[R4,#4]          ; get x->nxt_address
            CMP        R5, #0              ; Comparing with 0
            MOVEQ      R12,#1              ; Initialize l
            BEQ        Done
            STMFD      SP!,{R5}            ; passing x->nxt_address to stack
            BL        Length
X LDMFD      SP!,{R5}
            ADD        R12,R12,#1          ; return l + length (nxt_Address)
Done LDMFD      SP!,{R4-R5,LR}             ; Restore registers
            MOV        PC, LR              ; same as bx lr
```

SP

R4
R5
LR
0x124
R4
R5
LR
0x100

R4	R5	R12
0X124	0X140	2

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108



```
Length      STMFD      SP!,{R4-R5,LR}      ;Store regs to stack
            LDR        R4,[SP,#12]         ; read address of X
            LDR        R5,[R4,#4]          ; get x->nxt_address
            CMP        R5, #0              ; Comparing with 0
            MOVEQ      R12,#1              ; Initialize l
            BEQ        Done
            STMFD      SP!,{R5}            ; passing x->nxt_address to stack
            BL         Length
X LDMFD      SP!,{R5}
            ADD         R12,R12,#1          ; return l + length (nxt_Address)
Done LDMFD      SP!,{R4-R5,LR}              ; Restore registers
            MOV        PC, LR              ; same as bx lr
```

SP

R4
R5
LR
0x124
R4
R5
LR
0x100

R4	R5	R12
0X124	0X140	3

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108



```
Length      STMFD      SP!,{R4-R5,LR}      ;Store regs to stack
            LDR        R4,[SP,#12]         ; read address of X
            LDR        R5,[R4,#4]          ; get x->nxt_address
            CMP        R5, #0              ; Comparing with 0
            MOVEQ      R12,#1              ; Initialize l
            BEQ        Done
            STMFD      SP!,{R5}            ; passing x->nxt_address to stack
            BL        Length
X LDMFD      SP!,{R5}
            ADD        R12,R12,#1          ; return l + length (nxt Address)
Done LDMFD      SP!,{R4-R5,LR}              ; Restore registers
            MOV        PC, LR              ; same as bx lr
```

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108

SP

0x124

R4

R5

LR

0x100

R4

0X100

R5

0X124

R12

3



```
Length      STMFD      SP!,{R4-R5,LR}      ;Store regs to stack
            LDR        R4,[SP,#12]         ; read address of X
            LDR        R5,[R4,#4]          ; get x->nxt_address
            CMP        R5, #0              ; Comparing with 0
            MOVEQ      R12,#1              ; Initialize l
            BEQ        Done
            STMFD      SP!,{R5}            ; passing x->nxt_address to stack
            BL         Length
X LDMFD      SP!,{R5}
            ADD        R12,R12,#1          ; return l + length (nxt_Address)
Done LDMFD      SP!,{R4-R5,LR}              ; Restore registers
            MOV        PC, LR              ; same as bx lr
```

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108

SP

0x124

R4

R5

LR

0x100

R4

0X100

R5

0X124

R12

3



Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize l
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return l + length (nxt_Address)
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108

SP

R4
R5
LR
0x100

R4

0X100

R5

0X124

R12

3



Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize 1
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	LDMFD	SP!,{R5}	
		ADD	R12,R12,#1
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108

SP

R4
R5
LR
0x100

R4

0X100

R5

0X124

R12

4



Length	STMFD	SP!,{R4-R5,LR}	;Store regs to stack
	LDR	R4,[SP,#12]	; read address of X
	LDR	R5,[R4,#4]	; get x->nxt_address
	CMP	R5, #0	; Comparing with 0
	MOVEQ	R12,#1	; Initialize 1
	BEQ	Done	
	STMFD	SP!,{R5}	; passing x->nxt_address to stack
	BL	Length	
	LDMFD	SP!,{R5}	
	ADD	R12,R12,#1	; return 1 + length (nxt Address)
Done	LDMFD	SP!,{R4-R5,LR}	; Restore registers
	MOV	PC, LR	; same as bx lr

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108

SP

0x100

R4

0X100

R5

0X124

R12

4



```
Length    STMFD    SP!,{R4-R5,LR}    ;Store regs to stack
          LDR      R4,[SP,#12]      ; read address of X
          LDR      R5,[R4,#4]       ; get x->nxt_address
          CMP      R5, #0           ; Comparing with 0
          MOVEQ    R12,#1           ; Initialize l
          BEQ      Done
          STMFD    SP!,{R5}         ; passing x->nxt_address to stack
          BL       Length
          LDMFD    SP!,{R5}
          ADD      R12,R12,#1       ; return l + length (nxt_Address)
Done       LDMFD    SP!,{R4-R5,LR}  ; Restore registers
          MOV      PC, LR           ; same as bx lr
```

To calling subroutine
(e.g., main)

0x100

50

0x104

0x124

0x108

10

0x10C

0x000

0x124

54

0x140

0x140

-1

0x108

SP

0x100

R4

0X100

R5

0X124

R12

4